IBM QISKIT MISSION LAB

Q/ROUTE

Detailed project report

Interactive threat-aware drone routing with classical grid pathfinding and quantum visiting-order optimization.

16-qubit original IBM track

25-qubit fixed-depot experiment

React mission editor + FastAPI backend

Verified local CPU and Google Colab T4 results

Implementation report | 12 November 2026

Based on the implemented source code, README, automated checks, and stored execution records.

1. Project overview

Q/ROUTE is an interactive demonstration of quantum-assisted, threat-aware drone route planning. A user edits a mission containing a depot, delivery checkpoints, physical obstacles, and electronic warfare (EW) exclusion zones. Classical grid pathfinding computes safe travel paths. A Qiskit Quantum Approximate Optimization Algorithm (QAOA) circuit then samples candidate checkpoint visiting orders. The application validates those assignments, compares the best feasible sampled tour against exact classical enumeration, and animates a single drone along the corresponding grid paths.

The project combines a functioning optimization backend with a React mission editor and analysis interface. It runs locally without an IBM account, credentials, a paid service, or physical quantum hardware. An optional larger experiment has also been executed using Qiskit Aer GPU on a Google Colab NVIDIA T4.

The intended contribution is an understandable, inspectable integration of classical navigation and quantum optimization. It does not demonstrate quantum speedup, operational drone control, or a production military navigation system.

Implemented modes

Mode	Locations	Assignment variables / qubits	Path costs	Quantum execution
Original fixed-coordinate regression	Depot + 3 targets	16	Euclidean distance plus unsafe-edge penalty	Qiskit local statevector primitives
Interactive corridor mission	Depot + 3 targets	16	Safe Dijkstra grid-path lengths	Qiskit local statevector primitives
Experimental fixed-depot mission	Depot + 5 targets	25	Safe Dijkstra grid-path lengths	Optional Aer CPU or NVIDIA GPU

D-Wave and NVIDIA CUDA-Q are not implemented. NVIDIA execution in the recorded experiment uses **Qiskit Aer GPU**, which is a different framework from CUDA-Q.

2. Problem definition and scope

One drone must depart from depot 00, visit every delivery checkpoint once in the selected visiting order, and return to depot 00. It must follow navigable grid cells while avoiding walls, blocked cells, and EW exclusion zones. The objective is to minimize the sum of travel costs over the complete tour, including the final return leg.

A checkpoint is a required delivery location. Obstacles are blocked grid cells. An EW zone is a circular region excluded from safe grid navigation; its interpretation in the interface is GPS jamming or restricted airspace. These are mathematical constraints in a demonstration environment, rather than a model of actual radio propagation.

The mission is planar and static during computation. Movement is orthogonal, with no diagonal steps. There is one drone and no battery, payload, wind, altitude, flight dynamics, time windows, or multi-drone scheduling model. All targets have equal service priority.

3. System architecture

```
React / TypeScript mission editor
          |
          | environment, revision, solver settings
          v
FastAPI validation and solver endpoints
          |
          +--> Dijkstra: safe paths + recorded exploration
          |              + shared travel-cost matrix
          |
          +--> Exact enumeration: reference optimum
          |
          +--> Qiskit QuadraticProgram
                 -> QUBO -> Ising Hamiltonian
                 -> parameterized QAOA circuit
                 -> COBYLA expectation optimization
                 -> final sampled bitstrings
                 -> assignment decoding and safety checks
          |
          v
React results, analysis, search replay, drone playback
```

The boundary between classical and quantum responsibilities is explicit: **Dijkstra searches grid cells; QAOA optimizes the checkpoint visiting order.** A grid cell is not represented by one qubit. Increasing grid resolution changes pathfinding work and distances, while keeping the selected assignment encoding's qubit count unchanged.

The frontend communicates through HTTP requests. Vite forwards `/api` to the local FastAPI service during development. The backend constructs the matrix from the submitted environment rather than accepting stale frontend travel costs.

4. Grid environment and classical navigation

4.1 Demonstration geometry

The default grid is 40 x 40 cells; supported sizes range from 24 to 64. Its checkpoints occupy varied interior locations. Two baffles with staggered openings create corridors that interrupt direct connections and force turns. A circular EW region additionally removes traversable space.

At the default resolution, the six-location preset places checkpoints at cells `(6,10)`, `(20,12)`, `(20,29)`, `(33,25)`, `(5,31)`, and `(33,8)`. The four-location corridor preset uses the first four of these. Depot 00 is the first checkpoint.

The original corner-grid preset remains separate for regression coverage. The original standalone CLI also preserves its fixed Euclidean-coordinate scenario; it is distinct from grid navigation.

4.2 Dijkstra implementation

For each unordered pair of checkpoints, the backend runs deterministic Dijkstra search using a heap, distance table, predecessor map, and four-neighbor moves. It rejects out-of-bounds cells, obstacle cells, and excluded EW cells. It also tests movement segments against the threat circles.

The implementation records actual settled cells and newly discovered frontier cells at each exploration step. These records drive search playback. The renderer does not invent random exploration patterns or depict QAOA as a maze-search algorithm.

When the destination is reached, the predecessor map reconstructs a connected cell-by-cell shortest path. Four locations require six unordered pair searches; six locations require fifteen. Reverse journeys reuse the corresponding path in reverse.

4.3 Shared cost matrix

For a grid of size `s`, the implemented step length is:

```
step = 4 / (s - 7)
C[i,j] = (number of cells in path(i,j) - 1)  x  step
C[i,i] = 0
```

These are normalized demonstration distance units, not calibrated meters. The matrix is symmetric because movement costs and restrictions are direction independent. Both QAOA and exact enumeration consume this same matrix.

The route cost is the sum of matrix entries for every consecutive checkpoint pair, including the return to depot. The displayed flight path concatenates their actual grid paths. A shortest leg may pass through another checkpoint's cell, and legs may revisit corridor cells; the assignment model specifies a service visiting order rather than forbidding such physical pass-throughs.

4.4 Safety and unreachable environments

Editable missions conservatively block any grid cell whose square touches an EW circle. Segment-circle checks provide an additional geometric safeguard. A checkpoint covered by an obstacle or EW exclusion is rejected. If a checkpoint pair is unreachable, validation reports the unavailable connection instead of inventing a finite cost or drawing a direct replacement line. QAOA is skipped for an invalid mission.

For valid grid missions, every matrix edge refers to an already validated safe path. The grid renderer must follow those paths; it cannot replace them with straight checkpoint-to-checkpoint segments.

5. Original four-location QAOA formulation

5.1 Binary variables and constraints

Let `x[i,p]` be 1 when location `i` occupies tour position `p`, and 0 otherwise. There are four locations and four positions, producing **16 binary variables and 16 assignment qubits**.

The constraints are:

```
For every location i:  sum over p of x[i,p] = 1
For every position p:  sum over i of x[i,p] = 1
```

There are eight equality constraints. They enforce one appearance per location and one location per position.

5.2 Cyclic objective and QUBO

The travel objective is:

```
F(x) = sum over p,i,j with i != j:
       C[i,j]  x  x[i,p]  x  x[j,(p+1) mod 4]
```

The modulo term includes the closing tour edge. The code builds this model with Qiskit Optimization's `QuadraticProgram` and converts it using `QuadraticProgramToQubo`.

The implemented penalty adds 1,000 times the squared violation of each one-hot constraint. Valid assignments have zero assignment penalty. This penalty is separate from the original map's configurable **10,000-unit unsafe-edge travel penalty**. Neither penalty removes the need to validate a sampled route.

QUBO-to-Ising conversion uses the binary/Pauli correspondence `x = (1 - Z) / 2`. The conversion returns an Ising Hamiltonian and a constant energy offset. The plotted optimization history adds the offset back to represent QUBO expectation. It is not the distance of one specific sampled tour.

5.3 Circuit and parameter tuning

The original circuit uses Qiskit's `qaoa_ansatz`, a uniform initial superposition, and the standard X mixer. Each QAOA layer alternates cost evolution with mixing. With p layers, there are 2p adjustable angles; the default two-layer circuit has four.

SciPy's COBYLA optimizer adjusts these angles without requiring analytic gradients. The UI's **COBYLA max evaluations** setting limits objective evaluations. For example, 40 is an optimization budget, not 40 shots, qubits, or layers. Reaching that budget may stop optimization before convergence.

`StatevectorEstimator` computes exact simulated expectation values during optimization. After optimization, `StatevectorSampler` draws the final measurement samples. The 4,096-shot setting refers to final sampling; it is not 4,096 measurements at every parameter evaluation.

The UI supports one to three layers. More layers generally increase circuit work and parameter count, but do not guarantee a better sampled solution. QAOA layer count is different from gate-level circuit depth, which depends on circuit construction and decomposition.

5.4 Decoding and exact comparison

Qiskit displays bitstrings with the highest-index bit on the left; decoding reverses this display order before interpreting assignments. The decoder rejects empty or multiply occupied positions and repeated locations. A valid cyclic assignment is rotated so depot 00 appears first and is appended again at the end.

The application distinguishes the most frequent bitstring from the best feasible sampled route. The most frequent sample can be invalid. Feasible-shot count includes repeated occurrences of valid assignments, rather than counting only distinct bitstrings.

Exact comparison enumerates all `3! = 6` depot-first orders for four locations. This reference is reported independently and is never substituted for a failed QAOA sample.

6. Experimental six-location, 25-qubit mode

6.1 Fixed-depot encoding

A full six-by-six assignment would require 36 qubits. The experimental encoding fixes the depot outside the assignment matrix and assigns five targets to five positions, requiring **25 variables/qubits** and ten one-hot constraints.

Its objective includes three components:

```
Departure:  sum_i C[0,i]  x  x[i,0]
Internal:   sum_p,i!=j C[i,j]  x  x[i,p]  x  x[j,p+1]
Return:     sum_i C[i,0]  x  x[i,4]
```

Only target-to-target adjacency terms are quadratic. The departure and return terms are linear. Exact comparison enumerates `5! = 120` orders using the same fifteen grid-path distances.

6.2 Initialization and XY mixer

The experimental circuit initializes each target row in a one-excitation W state. A row-preserving XY mixer moves that excitation among positions. This preserves the constraint that each target has exactly one assigned position. It does not automatically enforce one target per column; column uniqueness remains an energy constraint and a decoding check.

The initialization is not a classical optimal route. The circuit still samples invalid assignments, and feasible output is not guaranteed. The earlier uniform/X-mixer experimental baseline produced no feasible samples and was not presented as a successful result. The original four-location circuit remains unchanged.

6.3 CPU and GPU execution

The experimental backend uses Aer `EstimatorV2` and `SamplerV2`, with exact expectation precision and double-precision statevector simulation. The CPU/GPU device must be selected explicitly. Unsupported GPU selection raises an actionable error; there is no silent CPU fallback.

A 25-qubit double-precision statevector alone occupies 512 MiB, with additional runtime memory required. Dense statevector memory grows exponentially. This implementation does not provide a practical 100-qubit simulation for a full ten-location assignment model.

The Colab notebook installs the optional GPU requirements and runs a standalone experiment from a supplied source ZIP. It does not automatically connect the local web interface to a remote Colab service.

7. User interface and visual design

7.1 Design direction

The implemented interface uses the approved white, pale-blue, charcoal/navy, and lime-green visual direction. Rounded glass surfaces, restrained shadows, spacious typography, and locally bundled Manrope fonts establish a coherent mission-lab identity. Lime highlights primary actions and selected routes; soft blue indicates exploration; a contrasting sand tone indicates the search frontier. Obstacles use muted gray, while EW styling distinguishes restricted space.

The landing page combines alpine imagery, a drone visual, a concise project statement, and an entry point into Route Lab. Mountain photography is decorative and does not represent the mission's actual terrain. The landing route illustration is explicitly labeled as a scenario preview rather than a solver result.

7.2 Main application areas

Area	Function
Mission	Project introduction, simulator connection status, and Route Lab entry
Route Lab	Editable mission grid, solver settings, validation, and playback
Results	Best feasible sampled tour, cost, safety, execution metrics, exact comparison
Analysis	Actual sampled bitstrings, feasibility statistics, expectation history, bound circuit
Technology	Explanations of the classical/quantum workflow and encoding

The API status indicator represents backend reachability. An offline state does not establish a solver failure; the frontend may simply be unable to reach FastAPI.

7.3 Mission editor

Users can move the depot and targets, paint blocked cells, draw rectangular obstacles, erase cells, add or remove EW zones, and edit zone coordinates and radii. Up to four EW zones are supported. Checkpoint overlap and blocked placement are rejected.

The editor supports pan, zoom, fit-to-view, undo/redo with up to 50 states, restoration of checkpoints or zones, obstacle clearing, and full environment reset. Whole obstacle rectangles are not movable objects; users erase and redraw them. Environment edits are held in browser memory and do not persist after refresh.

Every edit creates a new revision and invalidates old results. Validation recalculates connectivity and costs after edits. Stale responses are ignored. Editing is locked during an active solver request. Resetting the frontend stops its

playback and discards responses, but does not forcibly terminate computation already running in Python.

7.4 Search and flight animation

Search playback reconstructs explored and frontier cells from recorded Dijkstra events. Final routes are SVG polylines through actual cell centers, including corridor turns. When QAOA returns a feasible visiting order, its leg paths are concatenated without adding artificial diagonal shortcuts.

A single animated drone follows the stitched route, pauses at targets, and returns to depot. Replay Search, Replay Flight, pause/resume, speed controls, and visibility controls support judging. Replay reuses recorded computations and does not rerun quantum optimization. Playback speed changes the presentation, not measured solver time or the optimization outcome.

8. API and software organization

File	Responsibility
`api/main.py`	Request schemas, validation, endpoints, execution locking, result metadata
`ibm_qiskit/grid.py`	Presets, obstacles, EW checks, Dijkstra, path-derived costs
`ibm_qiskit/qiskit_map.py`	Original map geometry, route cost/safety, exact enumeration
`ibm_qiskit/qiskit_qaoa.py`	Four-location model, QUBO/Ising, QAOA, decoding
`ibm_qiskit/qiskit_fixed.py`	Fixed-depot encoding, W initialization, XY mixer, Aer execution
`frontend/src/main.tsx`	Main application, settings, results, analysis, navigation
`frontend/src/MissionGrid.tsx`	Grid rendering, interaction integration, search and flight views
`frontend/src/environment.ts`	Environment edits, history, revisions, placement checks
`frontend/src/useGridEditor.tsx`	Editor controls and pointer actions
`frontend/src/playback.ts`	Playback clock, frontier reconstruction, route stitching
`frontend/src/styles.css`	Shared visual system and responsive styling
`notebooks/Q_Route_25_Qubit_GPU.ipynb`	Reproducible Colab GPU workflow

Key endpoints include `/api/health`, `/api/grid`, `/api/mission/validate`, `/api/mission/run`, and `/api/simulator`. The original `/api/map`, `/api/classical`, and `/api/qaoa/run` remain available. Interactive schemas are exposed at `/docs`.

Mission requests carry size, revision, four or six nodes, obstacle cells, EW zones, and `pathfinding: "dijkstra"`. Run requests add optimizer configuration and, for experimental mode, the selected Aer device. A backend lock rejects concurrent solver requests with HTTP 409 to prevent overlapping expensive local simulations.

9. Verified experimental results

The following measurements are taken from stored execution records, not estimated timings. Both experiments used six locations, 25 qubits, one QAOA layer, eight COBYLA evaluations, 4,096 shots, and seed 7.

Metric	Local Aer CPU	Colab T4 / Aer GPU
Solver elapsed time	124.898407959 s	13.297091673 s
Circuit depth	63	63
Feasible sampled shots	142 / 4,096	142 / 4,096
Feasible-shot percentage	3.4668%	3.4668%
Best sampled tour	0 -> 4 -> 2 -> 5 -> 3 -> 1 -> 0	0 -> 4 -> 2 -> 5 -> 3 -> 1 -> 0
Best sampled cost	19.8787878788	19.8787878788
Exact optimum cost	19.8787878788	19.8787878788
Best route safe	Yes	Yes
COBYLA convergence	No: evaluation budget exhausted	No: evaluation budget exhausted

A sampled route matching the exact optimum and optimizer convergence are different claims. These runs found an optimal feasible sampled route despite the optimizer stopping without convergence. The reversed route reported by exact enumeration has the same cost because the matrix is symmetric.

These timings cover the solver routine, not package installation, Colab allocation, browser interaction, or animation. CPU and GPU hardware differ; these two observations are not a controlled benchmark. Both are classical simulations of quantum circuits, and neither establishes quantum computational advantage.

Evidence:

• CPU result JSON

• GPU result JSON

• GPU run notes and notebook link

10. Verification and reliability

The automated suite covers original map geometry, safe complete tours, QUBO constraints, assignment encoding/decoding, return-edge costs, grid path connectivity, obstacle/EW avoidance, unreachable missions, edited matrices, API revisions, fixed-depot models, row-preserving initialization/mixing, explicit GPU failure, and small real Aer CPU execution.

Frontend logic tests cover checkpoint editing, obstacle painting/erasure, revision invalidation, route stitching in both directions, six-location compatibility, playback timing, target stops, and reconstruction of actual search frontiers. The production build performs TypeScript checking before generating the Vite bundle.

Verification commands:

```
.venv/bin/python -m pytest -q
.venv/bin/python -m pip check
node --test tests/environment.test.mjs tests/playback.test.mjs
```

```
npm --prefix frontend run build
```

Verification rerun while preparing this report passed **39 Python tests in 8.84 seconds**, **9 frontend logic tests**, and the TypeScript/Vite production build. Dependency checking reported **no broken requirements**. An upstream Starlette/httpx test-client deprecation warning does not indicate a failed project test. Automated checks do not constitute complete end-to-end visual or pointer-interaction coverage.

11. Installation and demonstration

The README is the primary setup guide, including Windows equivalents, troubleshooting, dependency versions, and optional GPU setup. The tested local environment uses Python 3.13 and Node.js 24.

From the repository root:

```
python3.13 -m venv .venv
.venv/bin/python -m pip install -r requirements-api.txt
npm --prefix frontend ci
```

Start FastAPI in one terminal:

```
.venv/bin/python -m uvicorn api.main:app --host 127.0.0.1 --port 8000 --reload
```

Start React in another:

```
npm --prefix frontend run dev
```

Open `http://127.0.0.1:5173`. The default four-location mode needs no optional Aer installation. Use `requirements-aer.txt` for the local experimental CPU mode, or the supplied Colab notebook and GPU requirements for NVIDIA execution. CPU and GPU Aer distributions should not be installed together.

A useful demonstration is to run the corridor mission, inspect its safe route and analysis, replay search and flight, then draw a wall that disconnects the grid. The application should report an invalid mission and skip QAOA. Resetting the environment restores the valid scenario.

The original CLI remains available:

```
.venv/bin/python -m ibm_qiskit.qiskit_demo
```

12. Limitations and future work

The present implementation is a local hackathon demonstration with explicit approximate-solver failure reporting. QAOA may return no feasible sample, and a large penalty or longer run does not guarantee an optimal answer. Exact enumeration is inexpensive at the implemented problem sizes and provides a strong reference.

The simulator uses ideal statevectors rather than physical-device noise, calibration, or hardware connectivity. The optional GPU experiment improves the observed simulation runtime on one configuration, while retaining exponential dense-state memory growth. CUDA-Q, physical IBM execution, larger practical routing instances, D-Wave, and remote UI-to-Colab integration remain outside the implemented scope.

Potential extensions include mission persistence, broader browser interaction tests, alternative constraint-preserving encodings, carefully controlled simulator comparisons, and richer vehicle constraints. These are future work, not features claimed by this report.